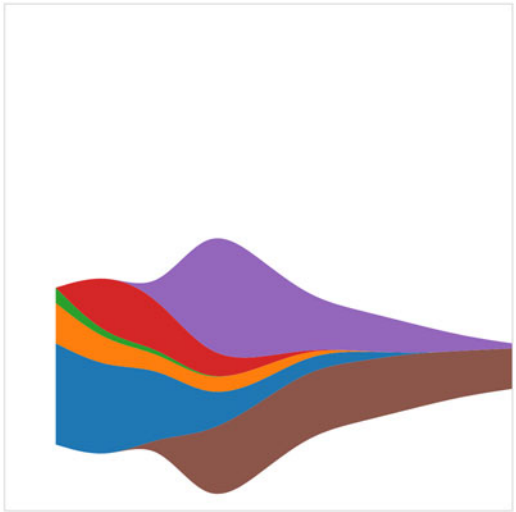


Figure 5.21 The streamgraph effect from a stack layout with basis interpolation for the areas and using the `silhouette` and `inside-out` settings for the stack layout. This is similar to our hand-built example from chapter 4 and shows the same graphical artifacts from the basis interpolation.

of the stacked areas around the middle. Another function useful for creating streamgraphs is the `.order()` function of a stack layout, which determines the order in which areas are drawn, so that you can alternate them like in a streamgraph. We'll use `inside-out` because that produces the best streamgraph effect. The last change is to the area constructor, which we'll update to use the `basis` interpolator because that gave the best look in our earlier streamgraph example:

```
stackLayout.offset("silhouette").order("inside-out");
stackArea.interpolator("basis");
```

This results in a cleaner streamgraph than our example from chapter 4, and is shown in figure 5.21.

The last time we made a streamgraph, we explored the question of whether it was a useful chart. It *is* useful, for various reasons, not least of which is because the area in the chart corresponds graphically to the aggregate profit of each movie.

But sometimes a simple stacked bar graph is better. Layouts can be used for various types of charts, and the stack layout is no different. If we restore the `.offset()` and `.order()` back to the default settings, we can use the stack layout to create a set of rectangles that makes a traditional stacked bar chart:

```
stackLayout = d3.layout.stack()
     .values(function(d) { return d.values; });

 var heightScale = d3.scale.linear()
     .domain([0, 70])
     .range([0, 480]);

 d3.select("svg").selectAll("g.bar")
     .data(stackLayout(stackData))
     .enter()
     .append("g")
```